INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

Seasons of Code 2026

END-TERM REPORT

Competitive Programming from First Principles

Name : Anjali Jogi

Mentors : Nishkarsh, Harshit

Institute : Indian Institute of Technology Bombay

Project : Competitive Programming from First Principles

Contents

Introduction

Competitive Programming is a discipline that focuses on developing algorithmic thinking, efficient implementation skills, and the ability to solve computational problems under strict time and memory constraints. It combines concepts from algorithms, data structures, mathematics, and logical reasoning to design optimized solutions for a wide variety of programming challenges. Regular participation in competitive programming strengthens analytical thinking, improves coding proficiency, and develops the ability to approach unfamiliar problems systematically.

The primary objective of the Summer of Science project, *Competitive Programming from First Principles*, was to build a strong foundation in competitive programming through a structured learning approach. The project began with the fundamentals of C++ Standard Template Library (STL), time complexity analysis, prefix sums, and difference arrays before gradually progressing to advanced topics such as greedy algorithms, binary search, bit manipulation, number theory, two pointers, sliding window techniques, recursion, backtracking, and dynamic programming.

Throughout the project, theoretical concepts were reinforced through extensive practice on competitive programming platforms including Codeforces, CSES, LeetCode, and AtCoder. Every week focused not only on understanding the underlying algorithms but also on recognizing common problem-solving patterns, analyzing constraints, selecting appropriate approaches, and implementing efficient solutions.

One of the major highlights of the project was the gradual transition from basic implementation problems to challenging contest-style questions. Continuous practice significantly improved problem-solving ability, debugging skills, implementation speed, and algorithmic intuition. Rather than memorizing solutions, emphasis was placed on understanding why a particular technique works and identifying situations where it can be applied effectively.

This report documents the complete learning journey undertaken during the Summer of Science program. It presents the concepts studied, practical problem-solving experience, important observations, challenges encountered, and the knowledge gained throughout the project from Week 1 to Week 7.

Chapter 1

Week 1: Foundations of Competitive Programming

The first week of the project was dedicated to building the fundamental knowledge required for competitive programming. Before solving algorithmic problems, it was essential to become familiar with the C++ Standard Template Library (STL), understand how algorithmic efficiency is measured, and learn basic techniques that frequently appear in programming contests. The primary objective of this week was to establish a strong foundation that would support the more advanced topics introduced in the later stages of the project.

1.1 Objectives

The major objectives for the first week were:

- Revise the fundamentals of the C++ Standard Template Library.
- Understand the importance of time complexity analysis.
- Learn Prefix Sum and Difference Array techniques.
- Develop confidence in solving basic implementation problems.
- Become familiar with competitive programming platforms and contest-style questions.

1.2 Revision of C++ Standard Template Library

The week began with a revision of the C++ Standard Template Library (STL), which forms the backbone of efficient competitive programming solutions. Commonly used containers such as vectors, pairs, maps, sets, queues, stacks, and deques were revisited along with frequently used algorithms provided by the STL.

Special attention was given to selecting appropriate data structures according to problem requirements while understanding the time complexity of their operations. This revision significantly improved coding efficiency and reduced the amount of manual implementation required during problem solving.

1.3 Time Complexity Analysis

Understanding the efficiency of an algorithm is one of the most important aspects of competitive programming. During this week, different complexity classes such as $O(1)$, $O(\log N)$, $O(N)$, $O(N \log N)$, and $O(N^2)$ were studied in detail.

Rather than only memorizing complexity formulas, emphasis was placed on analyzing algorithms based on the number of operations they perform for different input sizes. This understanding helped in identifying inefficient solutions and selecting optimized approaches that satisfy contest constraints.

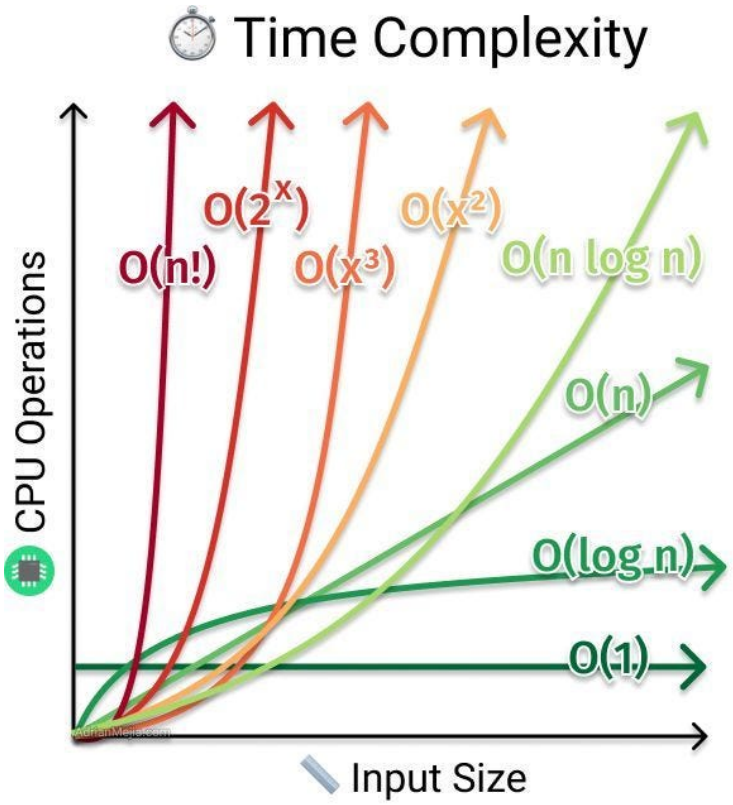

Figure 1.1: Time Complexity

1.4 Prefix Sums

Prefix Sum is one of the most fundamental preprocessing techniques used to answer range sum queries efficiently. Instead of recalculating the sum of every subarray repeatedly, cumulative sums are stored in advance, allowing each query to be answered in constant time after linear preprocessing.

Multiple implementation problems were solved to understand the practical applications of prefix sums, particularly in range query and subarray problems.

1.5 Difference Arrays

Difference Arrays were introduced as an efficient technique for handling multiple range update operations. Rather than updating every element inside a range individually, only the boundaries of the update are modified, after which the original array can be reconstructed using prefix sums.

This technique demonstrated how preprocessing can significantly reduce the complexity of problems involving repeated range modifications.

1.6 Problem Solving Practice

After completing the theoretical concepts, the focus shifted toward solving implementation-based programming problems. Mandatory problems from the assigned practice sheet covered arithmetic operations, conditional statements, loops, mathematical computations, and basic logical reasoning.

Additional problems from the second practice sheet were also solved to improve familiarity with contest environments and strengthen implementation skills. Continuous practice helped improve coding speed, debugging ability, and confidence while working under competitive programming constraints.

1.7 Summary

The first week established the fundamental concepts required for the remainder of the project. A strong understanding of STL, algorithmic complexity, Prefix Sums, and Difference Arrays provided the necessary groundwork for approaching more advanced algorithms in the following weeks. Regular problem solving also helped develop consistency in implementation and improved familiarity with the competitive programming workflow.

Chapter 2

Week 2: Developing Problem-Solving Skills

After building the fundamental concepts during the first week, the focus shifted towards applying these concepts to contest-style problems. The primary objective of this week was to develop logical thinking, observation skills, and implementation techniques required for solving Codeforces Div 2 A and B level problems. Rather than introducing advanced algorithms, emphasis was placed on strengthening problem-solving ability through extensive practice and recognizing common patterns that frequently appear in competitive programming contests.

2.1 Objectives

The major objectives for the second week were:

- Develop logical thinking through contest-style problems.
- Strengthen implementation skills using practical problem solving.
- Learn to identify common patterns in array and prefix sum problems.
- Gain experience with sorting and greedy-based techniques.
- Begin solving Codeforces Div 2 A and B rated problems independently.

2.2 Problem Solving Approach

A significant part of this week focused on understanding how experienced competitive programmers approach a problem before writing any code. Instead of immediately attempting implementation, importance was given to carefully reading the problem statement, analyzing the constraints, identifying useful observations, and constructing a logical solution.

This systematic approach greatly reduced unnecessary implementation attempts and improved the ability to design efficient algorithms before coding.

2.3 Prefix Sum and Frequency-Based Problems

Several practice problems involving prefix sums, cumulative frequencies, and subarray computations were solved during the week. Problems such as Two Sum, Subarray Sums, and Subarray Divisibility demonstrated how preprocessing and efficient data structures can significantly reduce computational complexity.

These exercises reinforced the practical applications of prefix sums while introducing the use of maps and hash-based containers for efficient counting and lookup operations.

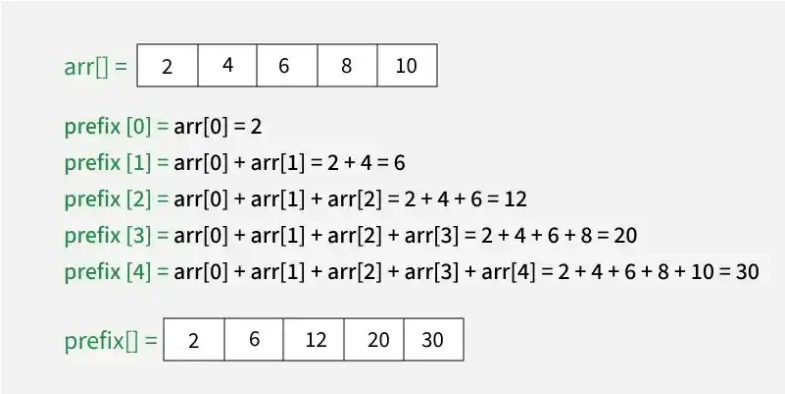

Figure 2.1: Prefix Sum

2.4 Sorting and Greedy Techniques

Sorting-based problems formed another major component of the week's practice. Various CSES problems required arranging elements strategically before performing efficient searches or greedy selections.

Problems such as Apartments, Ferris Wheel, Concert Tickets, Restaurant Customers, Movie Festival, and Sum of Two Values demonstrated how sorting often simplifies seemingly difficult problems by revealing useful ordering properties.

Alongside sorting, several introductory greedy problems were solved to understand how locally optimal decisions can often lead to globally optimal solutions when supported by correct reasoning.

2.5 Codeforces Div 2 Practice

After completing the recommended practice problems, regular practice on Codeforces was initiated using the rating filter. The focus remained primarily on Div 2 A and beginner-level Div 2 B problems.

These problems required careful observation, simple mathematical reasoning, constructive thinking, and clean implementation rather than complicated algorithms. Solv-

ing a variety of such problems helped improve familiarity with contest-style thinking and significantly increased confidence while approaching new problems.

2.6 Learning Outcomes

By the end of the week, problem-solving became considerably more structured. Instead of relying on trial-and-error approaches, solutions were developed by analyzing constraints, identifying patterns, and selecting suitable techniques before implementation.

Regular exposure to contest problems also improved debugging skills, implementation speed, and the ability to recognize recurring competitive programming patterns that appear across multiple problem categories.

2.7 Summary

The second week marked the transition from learning individual concepts to applying them effectively in competitive programming contests. Continuous practice across Codeforces and CSES strengthened logical reasoning, implementation ability, and confidence in solving beginner and intermediate contest problems. The experience gained during this week laid a strong foundation for studying more advanced algorithmic techniques in the following weeks.

Chapter 3

Week 3: Greedy Algorithms and Binary Search

The third week introduced two of the most important algorithmic paradigms used in competitive programming: Greedy Algorithms and Binary Search. Unlike the implementation-oriented problems solved in the previous weeks, these topics required deeper analytical thinking and mathematical reasoning to determine why a particular approach was correct. The primary objective was to understand when greedy decisions lead to optimal solutions and how binary search can be extended beyond searching in sorted arrays to solving optimization problems.

3.1 Objectives

The major objectives for the third week were:

- Understand the principles behind greedy algorithms.
- Learn exchange arguments and greedy correctness.
- Study Binary Search and Binary Search on Answer.
- Practice searching and sorting problems from CSES.
- Strengthen problem-solving skills through Codeforces greedy and constructive problems.

3.2 Greedy Algorithms

The week began with studying greedy algorithms, where locally optimal choices are made with the expectation of obtaining a globally optimal solution. Rather than memorizing standard greedy algorithms, emphasis was placed on understanding why greedy strategies work and identifying the conditions under which they are valid.

Exchange arguments and proof techniques were introduced to justify the correctness of greedy solutions. This helped in developing the ability to reason formally about algorithm correctness instead of relying solely on intuition.

Several greedy problems demonstrated how sorting, ordering elements appropriately, and making optimal local decisions can simplify problems that initially appear complex.

3.3 Binary Search

Binary Search was revisited from a broader perspective. Beyond searching for an element in a sorted array, the concept was extended to searching over the answer space of optimization problems.

The Codeforces EDU Binary Search lessons introduced the concept of *Binary Search on Answer*, where the search is performed over possible solution values instead of array indices. The feasibility of each candidate solution is verified using a separate checking function.

Understanding this technique proved particularly valuable because it is widely used in competitive programming contests for optimization problems involving monotonic properties.

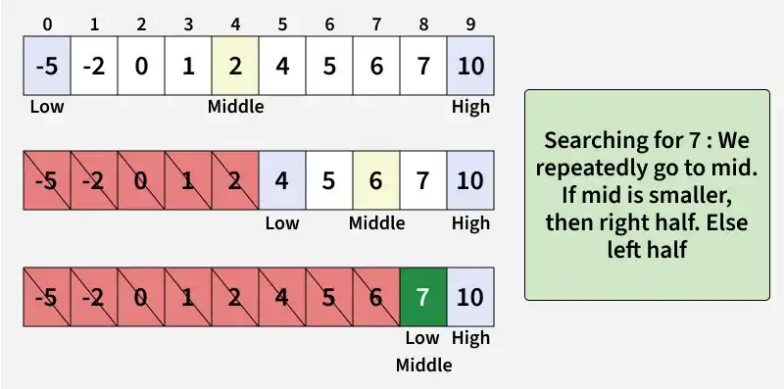

Figure 3.1: Binary Search

3.4 Searching and Sorting Practice

To reinforce these concepts, extensive practice was carried out using the CSES Searching and Sorting problem set. These problems required combining multiple ideas such as sorting, greedy observations, binary search, and efficient data structures to obtain optimal solutions.

Many problems emphasized identifying hidden monotonicity, maintaining sorted order, and reducing brute-force approaches to logarithmic complexity through binary search.

3.5 Codeforces Practice

Additional practice was performed using Codeforces problems filtered by the Greedy and Constructive tags within the 1000–1200 rating range. These problems encouraged observation-based thinking and demonstrated that many contest problems can be solved using relatively simple algorithms once the correct insight is identified.

Regular practice also improved implementation efficiency and strengthened the ability to quickly recognize recurring algorithmic patterns.

3.6 Learning Outcomes

The concepts covered during this week significantly enhanced analytical thinking. Instead of focusing only on implementation, greater emphasis was placed on proving algorithm correctness, identifying monotonic behavior, and selecting optimal strategies based on problem constraints.

The study of Binary Search on Answer and Greedy Algorithms introduced a new way of approaching optimization problems, making it possible to solve many problems with substantially lower time complexity.

3.7 Summary

The third week represented an important step toward advanced competitive programming. Greedy Algorithms and Binary Search introduced powerful techniques for solving optimization problems efficiently, while extensive practice strengthened reasoning ability, algorithm selection, and implementation skills. These concepts formed the basis for several advanced techniques that were studied in the subsequent weeks.

Chapter 4

Week 4: Bit Manipulation and Number Theory

The fourth week introduced two important areas of competitive programming: Bit Manipulation and Basic Number Theory. These topics are widely used in programming contests because they often provide elegant and highly efficient solutions to problems that appear difficult when approached using conventional methods. The objective of this week was to understand binary representations of numbers, learn common bitwise operations, and build a strong foundation in elementary number theory concepts that frequently appear in algorithmic problem solving.

4.1 Objectives

The major objectives for the fourth week were:

- Understand the fundamentals of bit manipulation.
- Learn common bitwise operations and their practical applications.
- Revise essential number theory concepts.
- Solve implementation and mathematical problems involving binary operations and prime numbers.
- Improve mathematical reasoning for competitive programming.

4.2 Bit Manipulation

The week began with studying the binary representation of integers and the use of bitwise operators such as AND, OR, XOR, NOT, and bit shifts. Rather than treating these operators as language-specific features, emphasis was placed on understanding how binary arithmetic can simplify computations and optimize algorithms.

Several common techniques were explored, including checking whether a particular bit is set, setting or clearing individual bits, counting set bits, determining powers of two,

and representing subsets using bitmasks. These operations provide efficient alternatives to many arithmetic computations and are frequently used in competitive programming.

The practice problems demonstrated how seemingly complex logical operations can often be simplified through careful manipulation of binary representations.

Operator	Example	Meaning
&	a & b	Bitwise AND
|	a | b	Bitwise OR
^	a ^ b	Bitwise XOR (exclusive OR)
~	~a	Bitwise NOT
<<	a << n	Bitwise left shift
>>	a >> n	Bitwise right shift

Figure 4.1: Bit Manipulation

4.3 Basic Number Theory

The second major topic of the week focused on introductory number theory. The concepts covered included divisibility, prime numbers, prime factorization, greatest common divisor (GCD), least common multiple (LCM), modular arithmetic, and fast exponentiation.

Instead of studying advanced mathematical theory, the focus remained on concepts that regularly appear in competitive programming contests. Understanding these topics made it possible to solve several mathematical problems more efficiently while avoiding unnecessary brute-force computations.

Particular attention was given to recognizing mathematical patterns and reducing computational complexity through appropriate number theoretic observations.

4.4 Problem Solving Practice

A variety of practice problems from Codeforces, LeetCode, CodeChef, AtCoder, and SPOJ were solved to reinforce the concepts studied during the week.

The bit manipulation problems required efficient use of binary operations to optimize implementations, while the number theory problems involved prime factorization, modular arithmetic, divisibility, and mathematical reasoning. These exercises highlighted

the importance of combining mathematical insights with algorithmic thinking to obtain efficient solutions.

Additional practice was carried out using Codeforces problem filters for the *bitmasks*, *number theory*, *math*, and *implementation* tags. Solving problems across different difficulty levels helped strengthen familiarity with these concepts and exposed recurring problem-solving patterns.

4.5 Learning Outcomes

This week's learning significantly improved the ability to interpret problems from a mathematical perspective. Bit manipulation demonstrated how binary operations can simplify implementation and improve efficiency, while number theory provided systematic approaches for solving computational problems involving divisibility, primes, and modular arithmetic.

The combination of theoretical understanding and practical problem solving enhanced analytical thinking and expanded the range of techniques available for tackling contest problems.

4.6 Summary

The fourth week strengthened both algorithmic and mathematical foundations by introducing Bit Manipulation and Basic Number Theory. These topics provided efficient computational techniques that are widely applicable in competitive programming and prepared the groundwork for more advanced algorithms studied during the remaining weeks of the project.

Chapter 5

Week 5: Two Pointers and Sliding Window Techniques

The fifth week introduced two of the most widely used techniques in competitive programming: the *Two Pointers* method and the *Sliding Window* technique. These approaches are fundamental for solving a large class of array and string problems efficiently by reducing unnecessary computations. Instead of relying on brute-force solutions with quadratic complexity, these techniques exploit the properties of contiguous sequences to achieve linear or near-linear time complexity. The primary objective of this week was to understand these patterns, identify situations where they are applicable, and strengthen implementation skills through extensive problem solving.

5.1 Objectives

The major objectives for the fifth week were:

- Understand the intuition behind the Two Pointers technique.
- Learn both Fixed Size and Variable Size Sliding Window approaches.
- Apply these techniques to array and string problems.
- Improve implementation efficiency by reducing brute-force solutions.
- Develop the ability to recognize common problem-solving patterns involving contiguous ranges.

5.2 Two Pointers Technique

The week began with studying the Two Pointers technique, where two indices are used to traverse a data structure while maintaining specific conditions. This approach is particularly effective for sorted arrays and problems involving pairs, triplets, or contiguous ranges.

Different variations of the technique were explored, including pointers moving towards each other, pointers moving in the same direction, and dynamically adjusting pointer positions based on problem constraints. Understanding these variations made it possible to

solve several optimization problems efficiently without examining every possible combination.

Through multiple practice problems, it became evident that many brute-force algorithms with quadratic complexity can be transformed into linear or logarithmic solutions by maintaining appropriate pointer movements and exploiting the structure of the input.

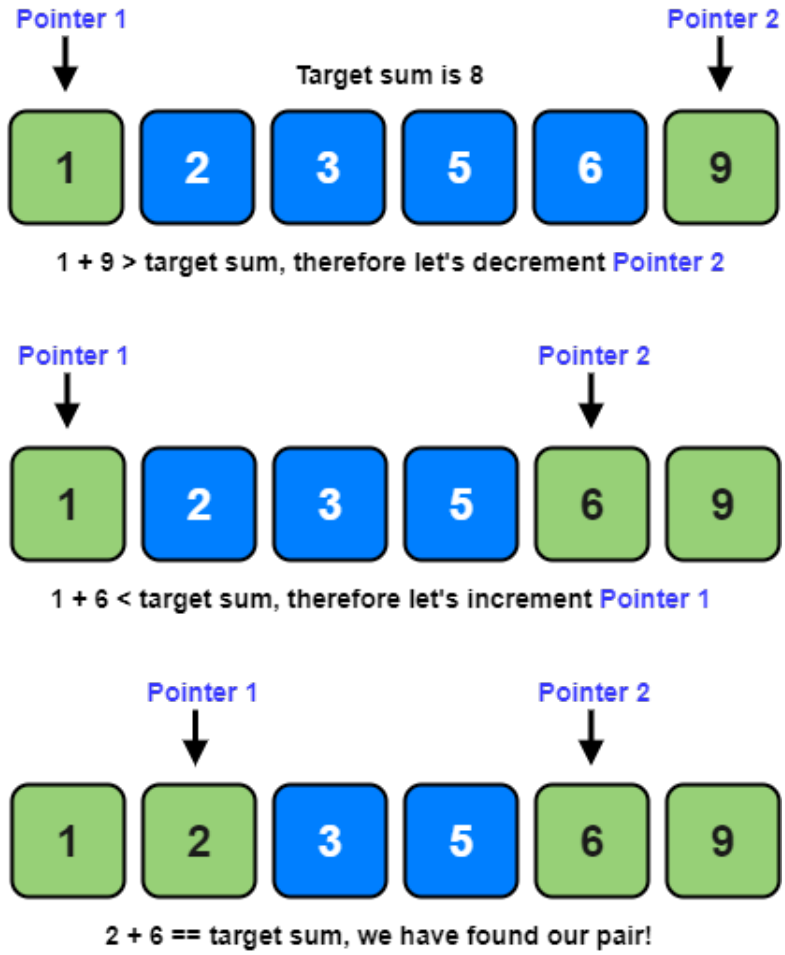

Figure 5.1: Two Pointers

5.3 Sliding Window Technique

The second major topic focused on the Sliding Window technique, which is widely used for solving problems involving contiguous subarrays and substrings. Instead of repeatedly recalculating information for every possible window, the current window is updated incrementally as it expands or contracts.

Both Fixed Size and Variable Size Sliding Window approaches were studied. Fixed-size windows were applied to problems where the window length remains constant, whereas variable-size windows required dynamically adjusting the boundaries to satisfy given constraints.

Special emphasis was placed on maintaining frequency tables, tracking distinct elements, and efficiently updating window information while avoiding unnecessary recomputation.

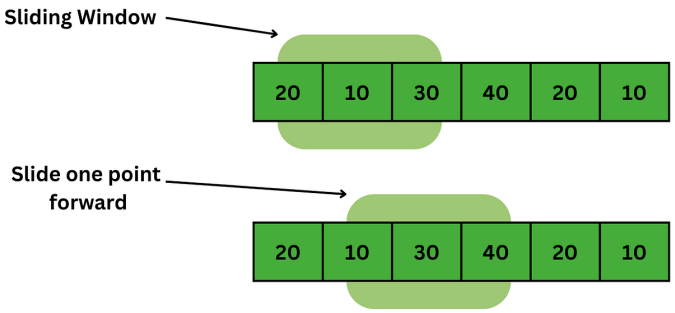

Figure 5.2: Sliding Window

5.4 Problem Solving Practice

A large number of practice problems from CSES, Codeforces, and LeetCode were solved throughout the week to reinforce these techniques.

Problems such as *Sum of Two Values*, *Sum of Three Values*, *Books*, *Kefa and Company*, and *Magic Powder* demonstrated different applications of the Two Pointers technique.

Similarly, Sliding Window concepts were strengthened by solving problems including *Playlist*, *Subarray Distinct Values*, *Longest Substring Without Repeating Characters*, *Minimum Size Subarray Sum*, *Fruit Into Baskets*, *Max Consecutive Ones III*, *Longest Repeating Character Replacement*, and *Fence*. These problems highlighted how maintaining an efficiently updated window enables optimal solutions for many array and string-based challenges.

Regular practice across these platforms improved implementation speed and helped identify recurring problem-solving patterns that frequently appear in competitive programming contests.

5.5 Learning Outcomes

This week significantly improved the ability to recognize problems that can be solved by maintaining dynamic ranges rather than examining every possible subarray. Understanding how to efficiently update pointers and maintain window properties led to substantial reductions in algorithmic complexity.

The experience gained from solving a diverse collection of practice problems also strengthened debugging skills, implementation accuracy, and confidence in applying these techniques to unfamiliar contest problems.

5.6 Summary

The fifth week introduced two of the most practical algorithmic techniques used in competitive programming. The Two Pointers method and Sliding Window technique provided efficient approaches for solving a wide variety of optimization problems involving arrays and strings. Extensive hands-on practice reinforced these concepts and prepared a strong foundation for the recursion and dynamic programming topics that followed in the subsequent weeks.

Chapter 6

Week 6: Recursion and Backtracking

The sixth week marked an important transition from iterative problem solving to recursive thinking. Unlike the techniques studied in the previous weeks, recursion requires expressing a problem in terms of smaller instances of itself. Building upon this idea, backtracking was introduced as a systematic approach for exploring multiple possible solutions while efficiently eliminating invalid search paths. The primary objective of this week was to understand recursive problem decomposition, visualize recursion trees, and develop complete search algorithms capable of solving combinatorial problems.

6.1 Objectives

The major objectives for the sixth week were:

- Develop intuition for recursive problem solving.
- Understand recursion trees and recursive state transitions.
- Learn the principles of backtracking and complete search.
- Practice generating permutations, combinations, and subsets.
- Solve classical recursion and backtracking problems from CSES, LeetCode, and Codeforces.

6.2 Recursion Fundamentals

The week began with studying recursion as a method of solving problems by repeatedly breaking them into smaller subproblems until reaching a base case. Rather than viewing recursion as simply a programming technique, emphasis was placed on understanding the recursive state, defining appropriate base cases, and visualizing the sequence of recursive calls through recursion trees.

Considerable attention was devoted to analyzing the time complexity of recursive algorithms and understanding how branching factors influence the total number of recursive calls. This helped build intuition regarding when recursive solutions are practical and when further optimization becomes necessary.

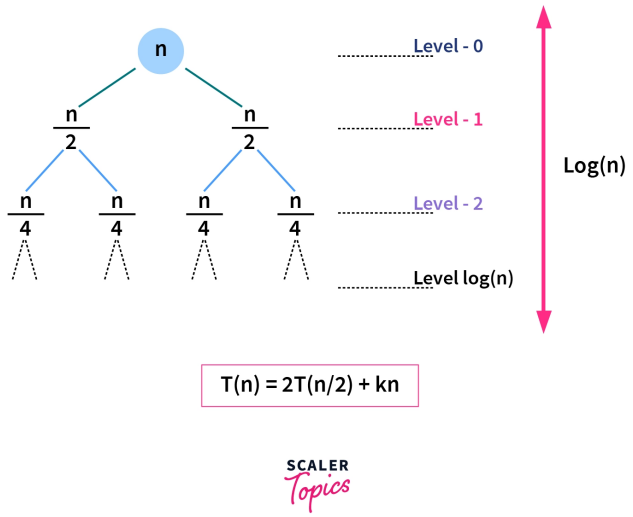

Figure 6.1: Recursion Tree

6.3 Backtracking and Complete Search

Building upon recursion, backtracking was introduced as an efficient strategy for exploring multiple candidate solutions while pruning invalid possibilities. Instead of exhaustively generating every possible configuration, constraints were applied during the search process to eliminate branches that could not contribute to a valid solution.

The concept of state-space exploration was studied through recursive search trees, where each recursive call represents a decision. Learning when to choose an option, recurse, and subsequently undo the choice before exploring the next possibility formed the foundation of effective backtracking algorithms.

This approach demonstrated how several combinatorial problems that initially appear computationally infeasible can be solved systematically through careful pruning and state management.

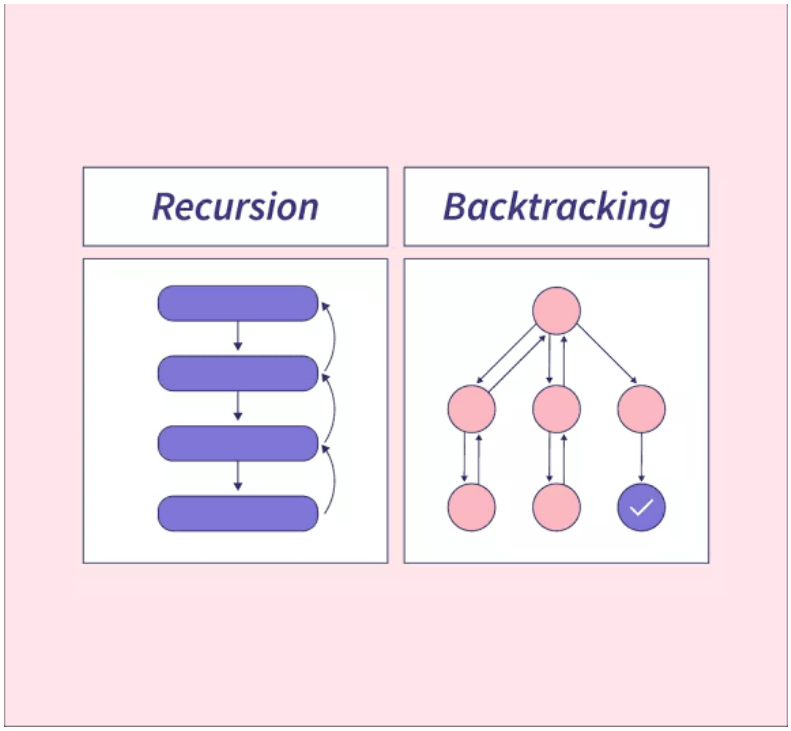

Figure 6.2: Recursion and Backtracking

6.4 Problem Solving Practice

A wide variety of recursion and backtracking problems were solved throughout the week to strengthen conceptual understanding as well as implementation skills.

The CSES introductory problems, including *Permutations*, *Creating Strings*, *Apple Division*, *Chessboard and Queens*, *Grid Paths*, *Gray Code*, and *Tower of Hanoi*, provided valuable experience in recursive problem decomposition and complete search.

Additional LeetCode problems such as *Subsets*, *Permutations*, *Combinations*, *Generate Parentheses*, *Palindrome Partitioning*, *Word Search*, *N-Queens*, *Sudoku Solver*, and the *Combination Sum* series further strengthened understanding of recursive state management and backtracking techniques.

Curated Codeforces recursion problems were also attempted to improve logical reasoning and implementation ability under contest conditions.

6.5 Learning Outcomes

This week significantly changed the approach toward solving search problems. Instead of attempting iterative solutions for every problem, recursive formulations became more intuitive through repeated practice and recursion tree analysis.

Backtracking introduced a structured framework for exploring large search spaces while avoiding unnecessary computations through pruning. As a result, confidence in

solving combinatorial problems and designing recursive algorithms improved considerably.

Furthermore, understanding recursion from first principles established a strong conceptual foundation for Dynamic Programming, where recursive solutions are systematically optimized to eliminate repeated computations.

6.6 Summary

The sixth week provided a comprehensive introduction to Recursion and Backtracking, two fundamental techniques that underpin many advanced algorithms. Through extensive practice across multiple online judges, recursive thinking, complete search strategies, and state-space exploration became significantly more intuitive. These concepts formed the conceptual bridge between brute-force recursive solutions and the optimized Dynamic Programming techniques introduced in the following week.

Chapter 7

Week 7: Dynamic Programming from First Principles

The seventh week introduced one of the most fundamental and powerful paradigms in algorithm design: *Dynamic Programming (DP)*. Unlike previous topics that focused primarily on greedy decisions or exhaustive search, Dynamic Programming emphasizes identifying overlapping subproblems and eliminating redundant computations. The primary objective of this week was not merely to learn standard DP algorithms, but to understand the reasoning behind state definition, state transitions, and the relationship between recursion and optimization.

7.1 Objectives

The major objectives for the seventh week were:

- Understand Dynamic Programming as an optimization of recursion.
- Learn how to identify overlapping subproblems and optimal substructure.
- Develop the ability to define meaningful DP states.
- Study memoization and tabulation approaches.
- Solve classical Dynamic Programming problems to strengthen problem-solving intuition.

7.2 Introduction to Dynamic Programming

The study of Dynamic Programming began by revisiting recursive solutions developed during the previous week. Instead of viewing DP as an entirely new concept, it was introduced as a systematic optimization of recursion, where previously computed results are stored and reused to avoid repeated calculations.

Considerable emphasis was placed on understanding the importance of identifying overlapping subproblems and recognizing situations where recursive calls repeatedly solve the same states. This perspective helped build a conceptual understanding of why Dynamic Programming significantly improves computational efficiency.

7.3 State Definition and State Transition

One of the most important aspects of Dynamic Programming is defining an appropriate state representation. Throughout the week, attention was focused on understanding what information a state should contain and why that information is sufficient to solve the remaining subproblem.

After defining the state, state transition equations were derived by analyzing how the current solution depends on previously solved states. Rather than memorizing recurrence relations, emphasis was placed on logically deriving transitions through careful observation of the recursive formulation.

This approach encouraged a deeper understanding of Dynamic Programming and made it easier to adapt known techniques to unfamiliar problems.

7.4 Memoization and Tabulation

Two standard implementation techniques for Dynamic Programming were explored during the week.

The first approach, *Memoization*, uses recursion together with a cache to store the results of previously solved states. This preserves the recursive structure while eliminating repeated computations.

The second approach, *Tabulation*, solves the same problem iteratively by computing smaller states first and gradually building the final solution. Comparisons between both approaches highlighted their respective advantages in terms of implementation style, recursion overhead, and memory usage.

Understanding both methods provided greater flexibility while designing efficient Dynamic Programming solutions.

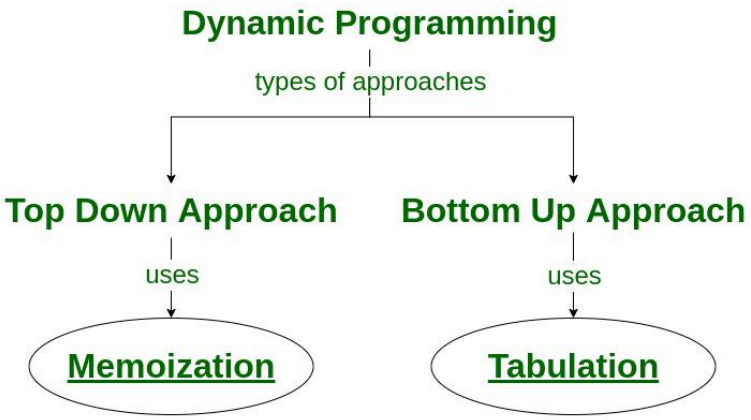

Figure 7.1: Dynamic Programming

7.5 Problem Solving Practice

The concepts studied during the week were reinforced through extensive practice on CSES and LeetCode.

Standard Dynamic Programming problems involving sequences, paths, subsets, and optimization were solved to understand different state representations and transition strategies. Each problem was approached by first developing a recursive solution, analyzing overlapping subproblems, and then transforming the recursion into an optimized DP solution.

Additional practice focused on recognizing recurring Dynamic Programming patterns rather than memorizing individual solutions. This helped build confidence in solving unfamiliar problems by reasoning from first principles.

7.6 Learning Outcomes

The study of Dynamic Programming significantly improved the ability to analyze recursive algorithms and optimize them systematically. Instead of viewing DP as a collection of formulas, it became clear that every solution originates from carefully defining a state and understanding how it relates to smaller subproblems.

Developing recursive intuition before optimization also made it easier to derive transition equations independently, reducing dependence on memorized patterns and improving overall problem-solving ability.

Regular practice strengthened confidence in identifying Dynamic Programming opportunities and implementing efficient solutions across a variety of problem categories.

7.7 Summary

The seventh week concluded the technical learning phase of the project by introducing Dynamic Programming from first principles. Through a combination of theoretical understanding and extensive practice, the concepts of state definition, state transitions, memoization, and tabulation became significantly more intuitive. The emphasis on reasoning rather than memorization provided a strong foundation for solving both classical and advanced Dynamic Programming problems encountered in competitive programming.

Challenges Faced

The project presented several challenges that contributed significantly to the overall learning experience. While the theoretical concepts were relatively straightforward to understand, applying them to unfamiliar contest problems often required extensive practice and careful analysis.

One of the initial challenges was developing the ability to identify the underlying pattern behind a problem. Many competitive programming problems can be solved using standard techniques such as Prefix Sums, Binary Search, Two Pointers, or Dynamic Programming, but recognizing the appropriate approach from the problem statement required considerable experience. This ability improved gradually through continuous exposure to a wide variety of problems.

Another major challenge involved optimizing brute-force solutions. During the early stages of the project, it was common to arrive at correct solutions that exceeded the time limits. Learning how to analyze time complexity, identify performance bottlenecks, and replace inefficient algorithms with optimized approaches was an important part of the learning process.

Recursive and backtracking problems also required a significant shift in thinking. Visualizing recursion trees, defining appropriate recursive states, and implementing effective pruning strategies initially proved difficult. However, consistent practice with standard recursion problems gradually improved confidence in designing recursive solutions.

Dynamic Programming was perhaps the most conceptually demanding topic of the project. Instead of memorizing recurrence relations, emphasis was placed on deriving state definitions and transition equations independently. This approach required deeper analytical thinking but ultimately led to a much stronger conceptual understanding of Dynamic Programming.

Another important challenge involved debugging complex implementations. Minor indexing mistakes, incorrect boundary conditions, and overlooked edge cases frequently resulted in incorrect outputs despite the overall approach being correct. Solving these issues improved debugging skills and reinforced the importance of systematic testing before final submission.

Overall, these challenges played a crucial role in strengthening logical reasoning, algorithmic thinking, and implementation skills. Each obstacle provided valuable learning opportunities that contributed to steady improvement throughout the duration of the project.

Overall Learning

The Summer of Science project provided a structured introduction to Competitive Programming by gradually progressing from fundamental concepts to advanced algorithmic techniques. Rather than focusing solely on solving problems, the project emphasized understanding the reasoning behind each algorithm, selecting appropriate techniques based on problem constraints, and developing efficient implementations.

Throughout the project, a strong foundation was established in several important areas, including:

- Efficient use of the C++ Standard Template Library (STL).
- Time complexity analysis and algorithm optimization.
- Prefix Sums and Difference Arrays for efficient preprocessing.
- Greedy Algorithms and Binary Search on Answer.
- Bit Manipulation and Basic Number Theory.
- Two Pointers and Sliding Window techniques.
- Recursive problem solving and Backtracking.
- Dynamic Programming using Memoization and Tabulation.

Extensive practice across platforms such as Codeforces, CSES, LeetCode, CodeChef, and AtCoder significantly improved implementation speed, debugging ability, and confidence while approaching contest problems. Solving problems of gradually increasing difficulty also strengthened observation skills and helped in recognizing recurring algorithmic patterns.

One of the most valuable outcomes of the project was the development of a systematic problem-solving approach. Instead of relying on trial-and-error methods, greater emphasis was placed on analyzing constraints, identifying useful observations, selecting suitable algorithms, and verifying correctness before implementation.

The project also reinforced the importance of consistency in competitive programming. Regular practice, careful review of editorials, and learning from unsuccessful attempts proved equally valuable as successfully solving problems. This mindset contributed to continuous improvement throughout the learning journey.

Conclusion

The *Competitive Programming from First Principles* project successfully established a strong foundation in algorithmic problem solving through a carefully structured progression of topics. Beginning with the fundamentals of C++ programming and algorithm analysis, the project gradually introduced advanced techniques including Greedy Algorithms, Binary Search, Bit Manipulation, Number Theory, Two Pointers, Sliding Window, Recursion, Backtracking, and Dynamic Programming.

Throughout the project, theoretical concepts were consistently reinforced through extensive problem solving on platforms such as Codeforces, CSES, LeetCode, CodeChef, and AtCoder. This practical approach significantly improved logical reasoning, implementation skills, debugging ability, and confidence in solving competitive programming problems under constraints.

Overall, the Seasons of Code project provided valuable experience in algorithmic thinking and efficient problem solving while establishing a strong foundation for future learning in advanced algorithms, data structures, and competitive programming contests.

Acknowledgements

I would like to express my sincere gratitude to my mentors, **Nishkarsh** and **Harshit**, for their continuous guidance, valuable feedback, and encouragement throughout the Seasons of Code project.

Their mentorship played a crucial role in developing a systematic approach towards competitive programming and in understanding both the theoretical concepts and practical techniques required for efficient algorithm design and problem solving.

I am also thankful to the **Seasons of Code (SoC), IIT Bombay** for providing an excellent platform to learn competitive programming through a structured curriculum and carefully designed practice resources.

Finally, I would like to acknowledge the developers and contributors of various competitive programming platforms and educational resources whose problem sets, tutorials, editorials, and documentation greatly contributed to my learning throughout this project.

References

1. CSES Problem Set — https://cses.fi/problemset/
2. Codeforces Problemset — https://codeforces.com/problemset
3. Codeforces EDU Courses — https://codeforces.com/edu/courses
4. LeetCode — https://leetcode.com/
5. CodeChef Practice — https://www.codechef.com/practice
6. AtCoder Problems — https://atcoder.jp/
7. CP-Algorithms — https://cp-algorithms.com/
8. USACO Guide — https://usaco.guide/
9. CSES Competitive Programmer's Handbook — https://cses.fi/book/book.pdf
10. Striver (take U forward) — Competitive Programming Resources.
11. Love Babbar — Competitive Programming and DSA Resources.